

Multivariate Time-Series Prediction Using Deep Learning

Appliance Energy Consumption Forecasting Using LSTM and GRU Models

Assessment Report

M.H. FATHIMA RISNA

BSc (Hons). Computer Science & Technology

Uva Wellassa University of Sri Lanka.

Date: 23rd May 2026

Table of Contents

1. Introduction
2. Dataset Insight
3. Preprocessing
4. Feature Engineering
5. Model Design
6. Results
7. Model Optimization:
8. Challenges and Solutions
9. Conclusion
10. References

1. Introduction

Energy consumption forecasting is an important problem in smart home and energy management systems. Accurate prediction of appliance energy usage helps optimize electricity consumption, reduce operational costs, and improve energy efficiency.

The objective of this project was to develop a deep learning-based time series forecasting system capable of predicting appliance energy consumption using environmental sensor data and historical appliance usage patterns.

Two deep learning architectures were implemented and compared:

- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

The project focused on:

- Data preprocessing
- Feature engineering
- Sequential data preparation
- Deep learning model training
- Hyperparameter tuning
- Performance evaluation

The final goal was to identify the most effective model for appliance energy forecasting.

2. Dataset Insights

The dataset used in this project contains appliance energy consumption measurements collected from a smart home environment. The data includes:

- Appliance energy usage
- Indoor temperature readings
- Humidity measurements
- Weather conditions
- Lighting information
- Time-based information

The target variable for prediction was:

`Appliances`

which represents the appliance energy consumption level.

The dataset contains multivariate time-series data recorded at regular intervals.

3. Preprocessing

- 1) Exploratory Data Analysis (EDA) was conducted to understand the characteristics of the dataset and identify important patterns.

Key Findings

3.1 Correlation Analysis

Correlation analysis showed that historical appliance consumption values had the strongest relationship with future energy consumption.

Strongly correlated features included:

Feature	Correlation
lag_1	0.75
rolling_mean_12	0.63
rolling_std_24	0.55
rolling_mean_24	0.51
lag_6	0.32
lag_12	0.31

These results indicated that past appliance usage patterns were highly predictive of future energy consumption.

3.2 Weak Features

Several features showed very weak correlation with the target variable, including:

- day
- month
- Visibility
- rv1
- rv2
- weak humidity sensors

These features were later removed during feature selection to reduce noise and improve model performance.

3.3 Time-Based Patterns

The dataset demonstrated clear temporal patterns:

- Higher energy usage during active hours
- Repeated daily consumption cycles
- Dependence on previous consumption behavior

2) Data Preprocessing, Several preprocessing steps were applied to prepare the dataset for deep learning models.

3.4 Missing Value Handling

Feature engineering operations such as:

- lag generation
- rolling window calculations

introduced missing values into the dataset.

These missing rows were removed using:

```
df = df.dropna()
```

This prevented numerical instability and NaN loss values during model training.

3.5 Feature Scaling

Min-Max Scaling was applied to normalize both:

- input features
- target variable

Scaling improved model convergence and stabilized GRU training.

Feature scaling formula:

$$\left[X_{\text{scaled}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}} \right]$$

Target scaling was especially important for improving deep learning model performance.

3.6 Train-Test Split

The dataset was split into:

- 80% training data
- 20% testing data

Shuffling was disabled to preserve time-series ordering.

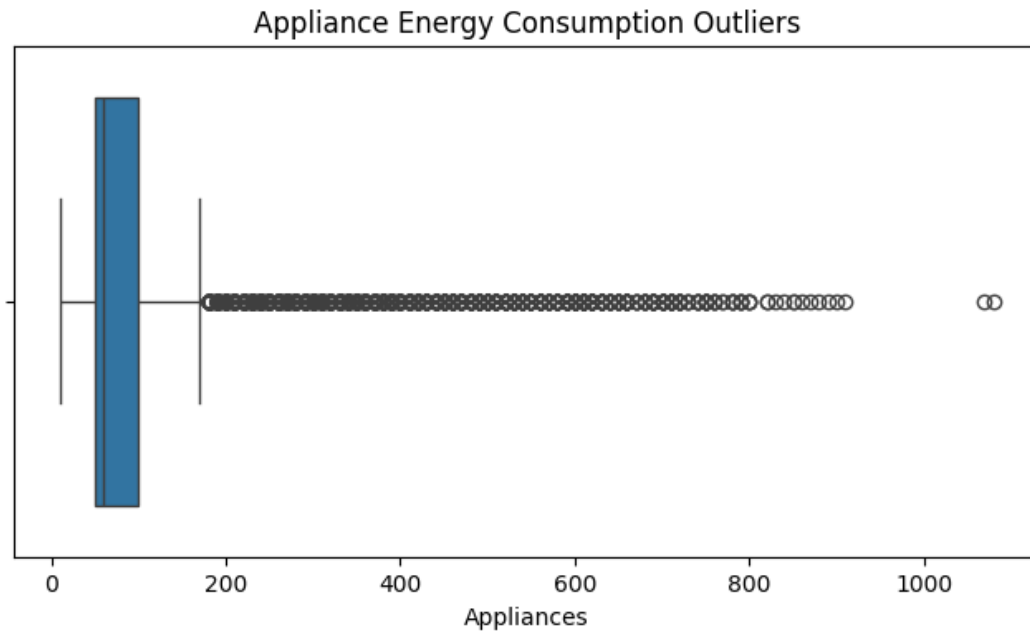


figure1: outlier

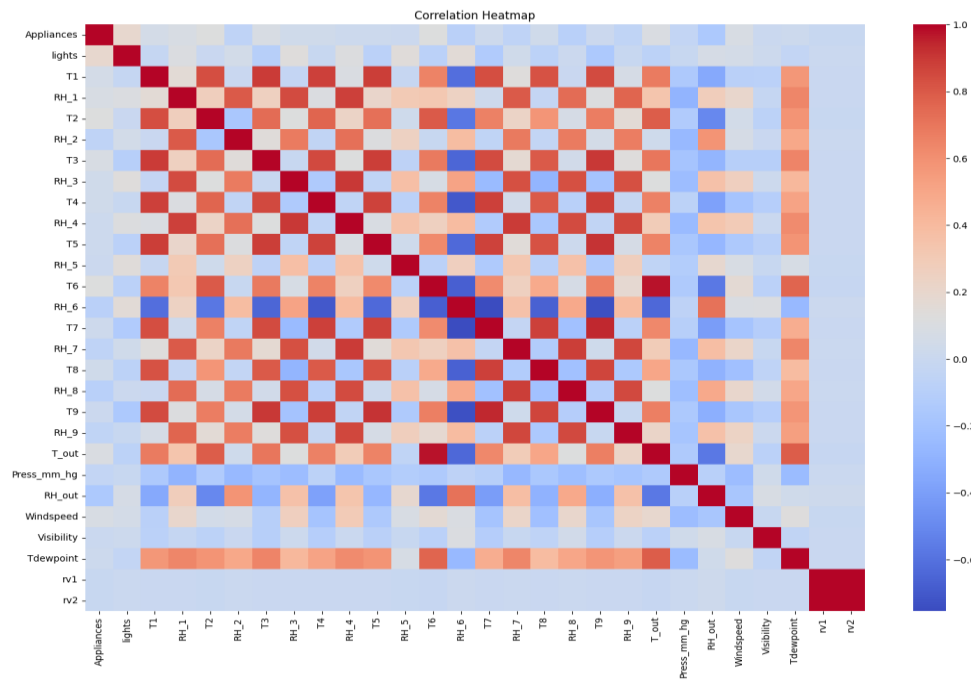


Figure2: correlation heatmap

4. Feature Engineering

Feature engineering significantly improved model performance.

4.1 Lag Features

Historical appliance consumption values were added as lag features.

Created lag features:

- lag_1
- lag_6
- lag_12
- lag_24

These features enabled the models to learn temporal dependencies from past energy usage.

4.2 Rolling Window Features

Rolling statistical features were generated to capture smoothed consumption patterns.

Created features:

- rolling_mean_12
- rolling_mean_24
- rolling_std_24

These features reduced noise and improved trend learning.

4.3 Time-Based Features

Additional temporal features included:

- hour
- lights

These features helped capture daily behavioral patterns.

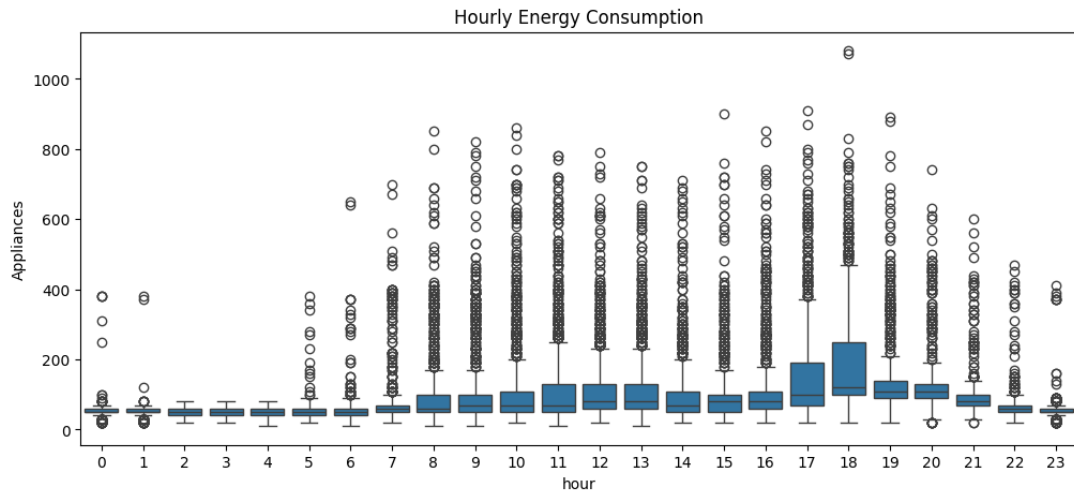


Figure3: hourly energy consumption

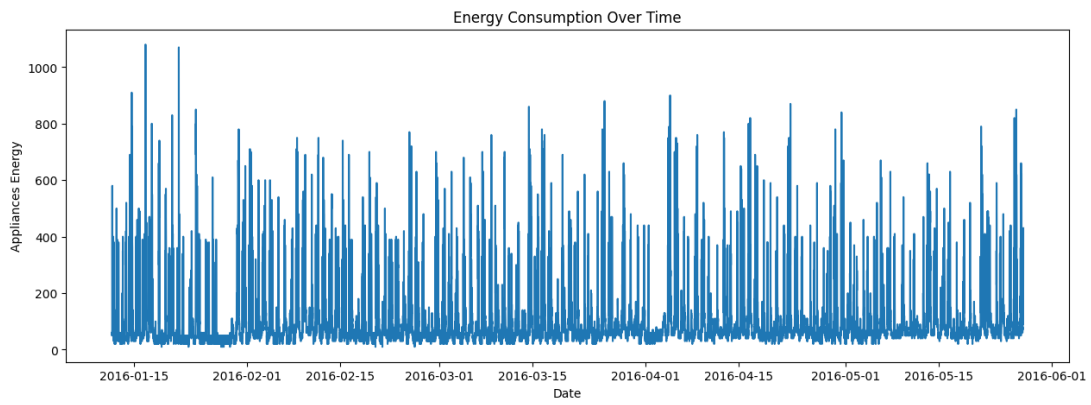


Figure4: energy consumption over time

4.4 Feature Selection

Weakly correlated features were removed based on correlation analysis.

Final selected features:

- lag_1
- lag_6
- lag_12
- lag_24
- rolling_mean_12
- rolling_mean_24
- rolling_std_24
- hour
- lights
- T2
- T6
- T_out
- temp_humidity
- Windspeed
- RH_1
- T3

Feature selection reduced model complexity and improved generalization performance.

5. Model Design

Two recurrent neural network architectures were implemented.

5.1 LSTM Model

The LSTM model architecture included:

- Two stacked LSTM layers
- Dropout regularization
- Dense output layer

Architecture:

LSTM(128) → Dropout → LSTM(64) → Dropout → Dense(1)

Activation Functions

- Default tanh activation for LSTM
- Linear activation for output layer

Optimizer

Adam optimizer was used due to:

- adaptive learning rates
- faster convergence
- stability in deep learning training

Loss Function

Huber Loss was selected because it:

- handles outliers effectively
 - improves robustness for noisy energy data
-

5.2 GRU Model

The GRU architecture was implemented similarly:

`GRU(128) → Dropout → GRU(64) → Dropout → Dense(1)`

GRU was chosen because:

- fewer parameters than LSTM
- faster training
- reduced computational complexity
- better performance without large dataset

The GRU model ultimately achieved better performance than LSTM.

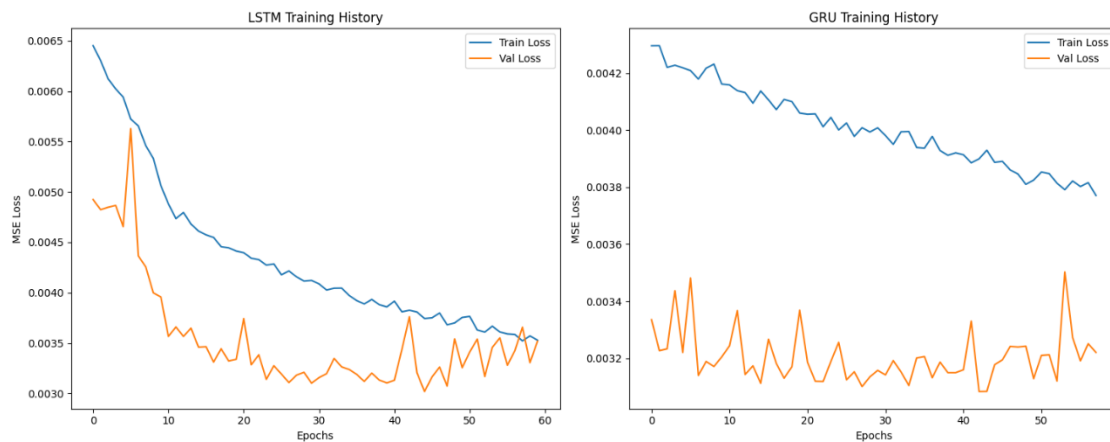


Figure5: LSTM vs GRU

6. Results

Final Model Performance

LSTM Results

Metric Value

MAE 35.75

RMSE 65.22

R² 0.448

GRU Results

Metric Value

MAE 30.51

RMSE 63.74

R² 0.473

Optimized GRU Results (Scaled Evaluation)

Metric Value

MAE 0.027

RMSE 0.059

R² 0.538

The optimized GRU model outperformed the LSTM model.

Predicted vs Actual Analysis

The GRU model successfully captured:

- overall consumption trends
- temporal patterns
- medium-scale fluctuations

However, sudden appliance spikes remained difficult to predict accurately

7. Model Training and Optimization

Several optimization techniques were applied during experimentation.

7.1 Early Stopping

Early stopping prevented overfitting by monitoring validation loss.

Configuration:

```
patience = 15
```

7.2 Batch Size Tuning

Different batch sizes were tested.

Final selected value:

```
batch_size = 32
```

This provided more stable gradient updates.

7.3 Learning Rate Adjustment

The learning rate was tuned to improve convergence stability.

Final learning rate:

```
0.0005
```

7.4 Sequence Length Tuning

Different sequence lengths were evaluated.

Tested values:

- 24
- 48

The best results were achieved using:

TIME_STEPS = 24

Longer sequences introduced additional noise and reduced performance.

7.5 Feature Reduction

Removing weakly correlated features significantly improved:

- model stability
- convergence
- generalization

This was one of the most impactful optimization steps.

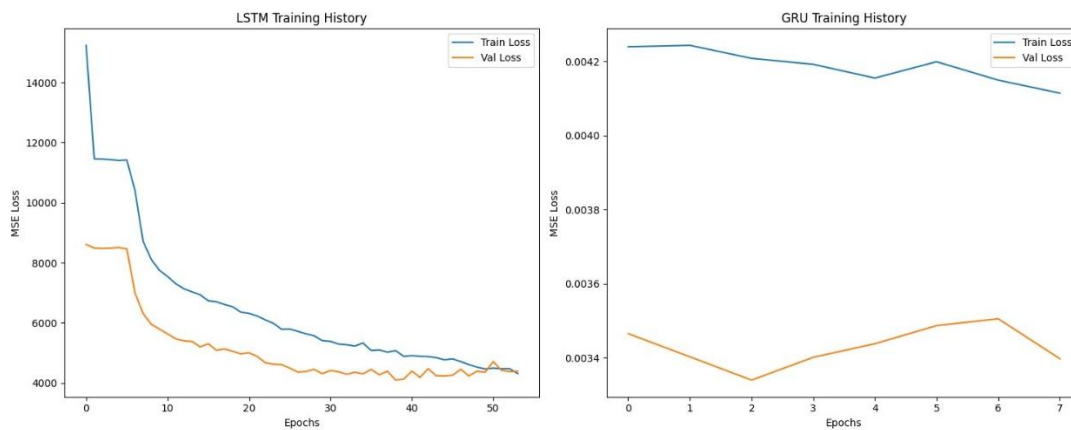


Figure6: LSTM vs GRU (after optimized)

8. Challenges and Solutions

Challenge 1: Identical LSTM and GRU Results

Initially, both models produced identical metrics.

Cause

Prediction variables were accidentally reused.

Solution

Separate:

- model objects
- prediction variables
- training workflows

were implemented for each architecture.

Challenge 2: NaN Loss Values

The model produced:

```
loss = nan
```

Cause

Lag and rolling features created missing values.

Solution

```
df = df.dropna()
```

was applied after feature engineering.

Challenge 3: Over-Complex Architecture

Larger GRU architectures reduced performance.

Solution

A simpler architecture with:

- fewer layers
- moderate dropout
- selected features

improved generalization.

Challenge 4: Weak Features

Many features had near-zero correlation with the target variable.

Solution

Correlation-based feature selection was applied to remove noisy inputs.

9. Conclusion

This project successfully developed and evaluated deep learning models for appliance energy forecasting.

Key achievements:

- Implemented LSTM and GRU architectures
- Performed feature engineering and sequence modeling
- Applied preprocessing and optimization techniques
- Improved model stability and forecasting performance

Among the evaluated models, GRU demonstrated better performance due to:

- faster convergence
- lower computational complexity
- improved generalization

The project highlighted the importance of:

- feature engineering
- data preprocessing
- proper sequence construction
- model simplification

in time-series forecasting tasks.

Future work could explore advanced hybrid architectures such as CNN-LSTM models, which combine convolutional layers for local pattern extraction with recurrent layers for long-term temporal learning. Such architectures may further improve energy consumption forecasting performance.

10. References

1. TensorFlow Documentation
2. Keras Documentation
3. Scikit-learn Documentation
4. Appliance Energy Prediction Dataset
5. Deep Learning for Time Series Forecasting Research Papers
6. Min-Max Scaling and Feature Engineering Tutorials